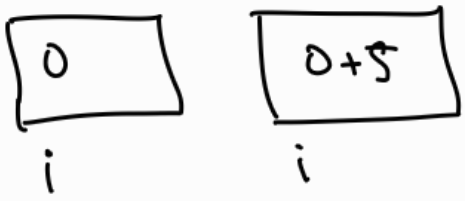
Repetition!

While loop → entry condition loop. It only runs when while(condition) → evaluates to true

```
i = 0;
while ( i <= 20)
  {
     cout << i << " ";
       i = i + 5;
  }
cout << endl;
```

first Run



output

0 5

note the loop control variable must be properly intialised before the while loop condition. In this case it is

i = 0

e.g you know how many times a certain statement needs to be executed.

↳ use while loop.

```
counter = 0;

while (counter < N)
{

    Statements;
    counter = counter + 1;

}
```

Runs N times.

if

counter = 1;

then condition is

counter <= N

inorder to Run N times.

(Sentinel controlled while loops)

You don't always know how many iterations need to be Run but you know the last iteration (entry aka Sentinel.



loop control variable is first item.

Int i = 1 (LCV)

while (i != Sential/last value)

{

Statements

}

// Intialise the loop control Variable

while (expression) // expression test the LCV

{

.
.
.

// Update the loop control Variable

.
.
.

}

In case of Sentinal loop the loop control Variable is Read First before while loop body.

```
cin>> variable
while ( variable != Senital)
{
    ⋮
    cin>> variable
    ⋮
}
```

Updates the LCV

Note

counter=0 is Intialised with zero at the Start because

no inputs are entered by user at start of program!!!

Note::

if you know how many times a loop will run. use counter control while loop which Basically means::

```
counter=0;
while (counter<N) {
  ++counter;
}
```

N times executed.

→ initialised to zero as no value has been taken in as input.

In Sentinal controlled while loop we always taken the first input Before the Body of the loop.

(flag controlled loops) → uses a bool variable to control loop

```
found = false;
while ( ! found) // test the loop control variable
   {

        if (expression)
           found = true; // update the loop variable
   }
```

(Number guessing game)

A program Randomly generates number b/w 0 (inclusive) and 100 (not inclusive). The program then prompts the user to guess the number. If the user corrects guess the number output appopriate msg if guessed number < random no. output (Low) and vice versa.

The user is prompted to guess the Random number until user enters the correct number.

To generate Random number

#Include <cstdlib>

rand() → (generates positive values) returns a int value b/w 0 and 32767

()

However each time the program runs, the rand() function generates the same Random numbers.

To generate diff random numbers each time you use the function Srand of header file cstdlib.

```
Srand (time (0)) ;
num = rand () % 100 ;
```

generates num b/w 0 and 99.

to include time use #include <ctime>

(for loop)

for (intial state; loop condition; update state)

↓

loops runs as long as condition remains true



goes to the loop body

}

(Imp Note)

```
for (int i=0; i<=4; i=i+1)
{
    cout<<"hello"<<endl;
    cout<<" *"<<endl;
}
```

output

hello
*
.1

hello
*
.2

hello
*
.3

hello
*
.4

hello
*
.5

```
for (int i=0; i<=4; i=i+1)
    cout<<" hello"<<endl;
    cout<<" *"<<endl;
```

output

hello -1
hello .2
hello .3
hello -4
hello .5
*

In a for loop if the loop condition is omitted, it is assumed to be **true**.

for(int i=0; ; i=i+1)
cout << i;

prints infinite no

```
for (;;)
    cout << "hello" << endl;
```

→ Infinite loop

```
for (int i=7; i<=10; i=i+1) ;
    cout << "hello" << endl;
    cout << " * " << endl;
```

Output
hello
*

Infinite loop

```
for(int i=0; i=i+1)
    cout << hello;
```

Removing the loop condition makes program go into infinite loop

```
for (int i=0; i<=9; i=i+1) ;
{
    cout << "hello" << endl;
}
```

← loops terminated

Output = hello . only one

Do while loop

```
do
    Statement
while (expression );
```

The Statement is executed first then the expression is evaluated. If the expression evaluates to true the Statement is again executed.

```
i=0;
do
{
  cout << i<<endl;
      i=i+5
}
while (i<= 20);
```

output

0
5
10
15
20

(Do while loop will run atleast one time)

```
i= 11 ;
while (i<=10)
 {
    cout << "hello";
    i=i+5;
 }
```

(No output)

```
i=11;
do
{
  cout << i<<endl;
  i=i+5;
}while (i<=10);
```

output: 11

A do-while loop can be used for input validation.

(Choosing the right loop)

→ If no of repetition is known then for

→ if you don't know no. of repetitions use while loop

→ if you don't know the no of repiplations But do know

that loop runs atleast 1 time in do-while.

Important point to remember:.

Write a program that ends with a Sentinal value and calculates the average of n positive and negative numbers

```
Int count=0
float sum=0, num;
cout<< " Enter a positive number:" <<endl;
cin>> num;
while (num != -999)
{
    sum = num + sum;
    count ++;
    cin>>num;
}
cout << "Average is :" << sum/count<<endl;
```

cin>>num; ← Don't write over here

note we are updating num value only after it has taken in and calculated the first value

```
#Include <ctime>
#Include <cstdlib>

Srand(time(0))
num = rand() % 50 → generates Random no from 0 till 49
num2 = rand() % 50 + 1 → generates Random no from 1 to 50.
```

fibonacci Series → two numbers fixed for inputs.

1 1 2 3 5

1+1=2 1+2=3 2+3=5

all this is done in loop

Take Input for number 1 and number 2.

→ first print the first number in program → num1.

→ Then add num1 + num2 in another variable e.g reg.

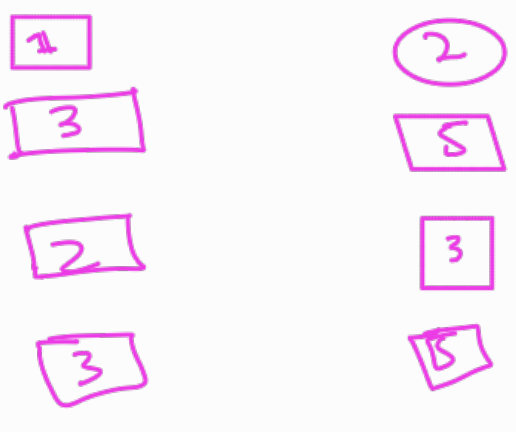
→ Assign num1 value of num2 and num2 value of reg.

→ Increment i if you are using while loop

→ e.g 1 1 2 3 5

```
num1=1
num2= 1
reg;
cout<< num1;
reg= num1 +num2
num1= num2;
num2 = reg;
```





(Factorial program) → factorial does not exist for negative no.s.

$4! = 1 \times 2 \times 3 \times 4$

$0!$	1
$1!$	1

runs till the no. e.g if you want to find 3! then program runs 3 times

```
int fact=1;
for (int i = 1; i<=num; i=i+1)
{
    fact = fact x i;
}
```

Divisibly test by 3 and 9

```
int num, res, Sum=0;
cout<<" Enter a num:" <<endl;
cin>> num;

while (num>0)
{
    res= num%10;
    Sum = Sum +res;
    num = num /10;
}
if (Sum%3==0 && Sum%9==0)
   cout<<" follows divisiblity";
else
   cout<<" Does not follows";
```

→ Calculates Sum of each digit for a n number.

Break and continue Statements in CPP

Break can be used in Switch, Loops of any type

If we use break in Loop it ends and exits the loop. In Switch Break skips the remainder of Switch statement.

```
for (int i=0; i<4 ; i++)
    {
        cout<<i<<endl;
        if (i==2)
            Break;
    }
```

→ exits the loop as soon i=2

Continue → Skips the current iteration.

```
for (int i=0; i<4 ; i++)
    {
        if (i==2)
          continue;
        cout<<i<<endl;
    }
```

← Skips the current iteration when i=2 and moves on.

output

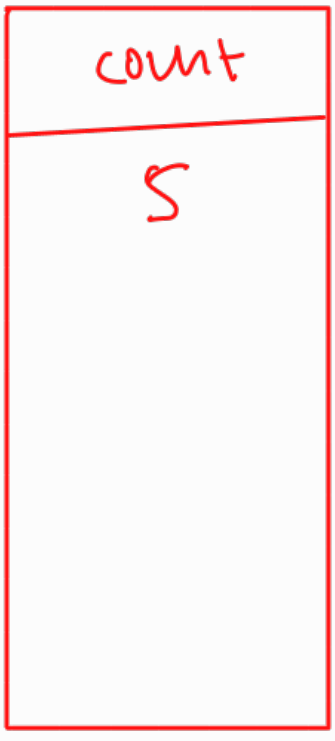
0
1
3
4
5
6
⋮
39

→ Input Valid
→ Device all program
→ mid + labs
→ Switch

What is the output of the following code:

```
int count = 5;
while (count-- > 0)
    cout << count << " ";
count<<endl;
```

5 > 0
4 > 0
3 > 0
2 > 0
1 > 0
0 > 0



output




```
int count = 5;
while (--count > 0)
    cout<< count << " ";
count<<endl;
```

output

4 3 2 1

note:

1 % 10 → 1

1 / 10 → 0

$$10 \overline{)\,1\,}\ 0 \quad -\ 0 \quad 1$$

Program to reverse a given Integer (Positive)

```
Int num, rem;
cin>> num;

while (num > 0)
{

    rem= num % 10;
    num= num /10;
    cout<< rem ;

}
```

Input 4321

Output 1234

But now I want to make the reversed num store any where.

```
Int num, rem=0;
while (num >0)
{

    rem= rem *10;
    rem= num % 10 + rem;
    num= num /10;

}
cout<< rem;
```

lets say user 123 enters

	1st	2nd	3rd
Run	0	30	320
	3	32	321
	12	1	0

→ now can store the reversed number anywhere.